DAY 10 · THEORY HANDOUT

RNN: Read Text in Order

Recurrent Neural Networks: carry information from one part of a sequence to the next.

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

1. Order changes the meaning

A **sequence** is a list where the order matters. Words in a sentence form a sequence.

Same words

"dog bites person"

The dog does the biting.

Different order

"person bites dog"

The person does the biting.

A model that only counts words loses their order. A **Recurrent Neural Network (RNN)** reads a sequence one step at a time.

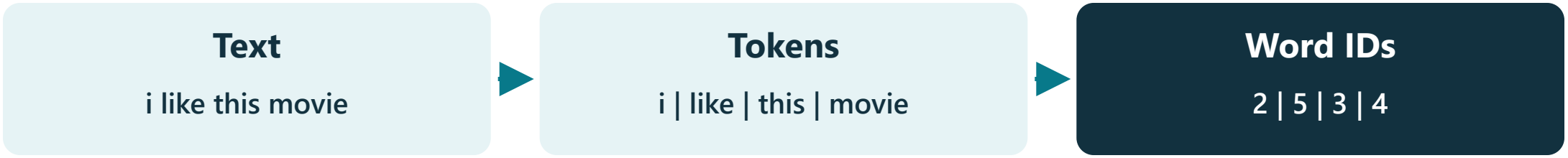
Natural Language Processing (NLP) means working with human language using computers. NLP existed before RNNs. RNNs became one way to model word order.

> We use an RNN when earlier steps can help explain the current step, such as words, sounds, or measurements over time.

2. Text becomes numbers

Computers calculate with numbers, so we first give each known token an ID.

Token = one text piece. Vocabulary = the list of known tokens.



A character model could use c | a | t. Our project uses whole words.

Tokenization splits text into pieces. A token can be a character, a word, or part of a word. Our simple example splits at spaces.

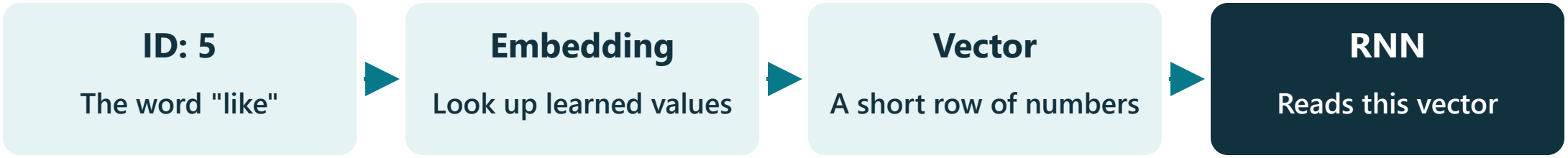
A **vocabulary** maps each known token to an integer ID. These example IDs are made up. ID 5 is not "more positive" than ID 2.

> A word ID identifies a token. It does not explain its meaning. An unknown word needs a special unknown-token ID.

3. Embedding gives each token a vector

An **embedding** maps a token ID to a learned row of numbers, called a **vector**.

Each token gets a vector, not one human meaning number.



i like this movie > four token vectors, read in order

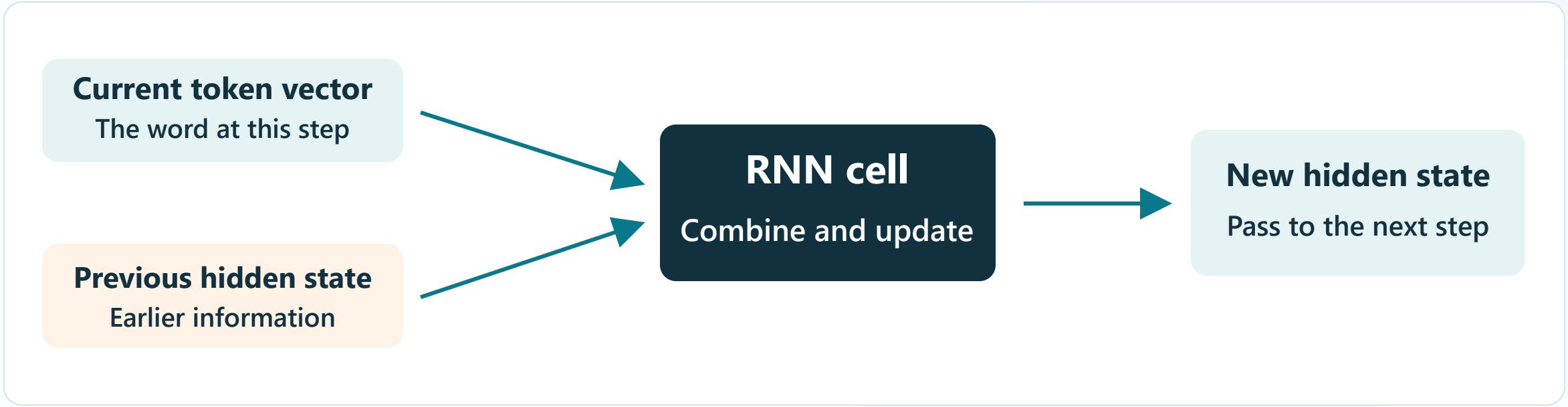
Embedding values start small and change during training. They can capture relationships useful for this task. They are not dictionary definitions.

Padding adds zeros to shorter ID lists so a batch has equal lengths. A **mask** tells the RNN to skip these added positions.

> Our notebook uses 0 for padding and mask_zero=True. A padding zero is not a real word.

4. The RNN cell and hidden state

A **hidden state** is a small vector that carries information from the earlier steps.



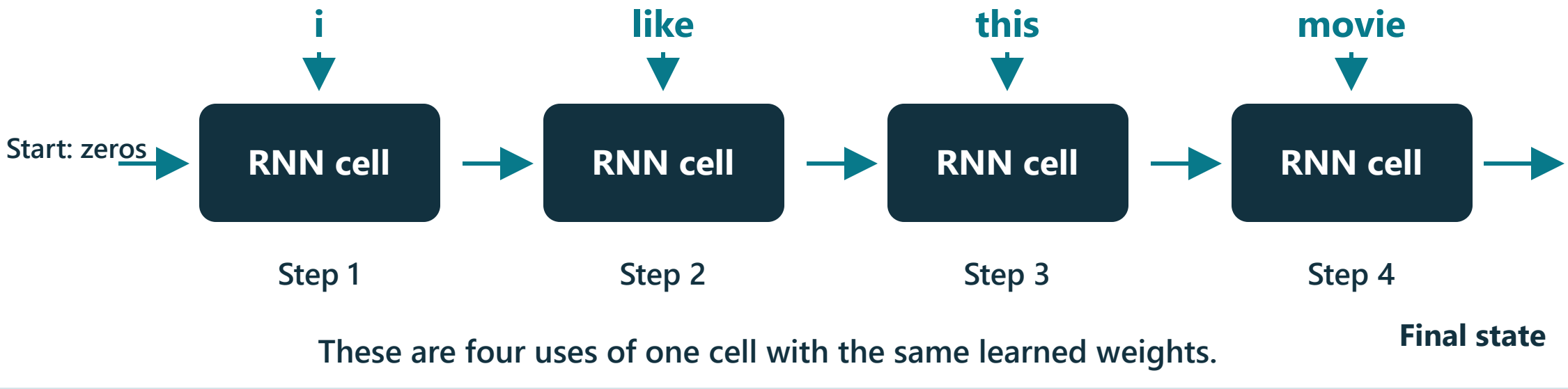
An **RNN cell** is the repeated calculation. It combines the current input and previous state using learned **weights**, numbers that control each contribution.

It adds a learned **bias** and applies an **activation**, a rule that changes the result. SimpleRNN uses tanh, which keeps each value between -1 and 1.

> The state is learned numbers, not a stored sentence. At the start of each independent sentence, we begin with a zero state.

5. One cell, shown across time

An **unrolled RNN** draws a separate box for each time step so we can see the order.



The first step reads "i". The next step reads "like" and receives the earlier state. The process continues until the last real word.

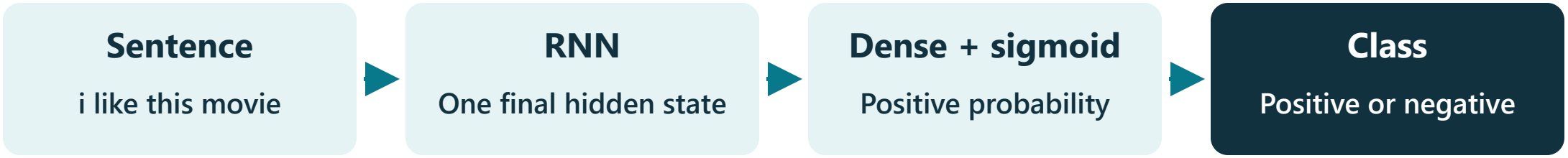
The boxes are the **same cell reused**, not four different trained models. The final state can hold patterns from the whole sentence.

> We do not label a hidden-state value "likes movie". The numbers only become useful through learning.

6. From the final state to a class

For this project, one sentence gives one label. This is a **many-to-one** task.

Sentiment classification = choose the feeling expressed in the text



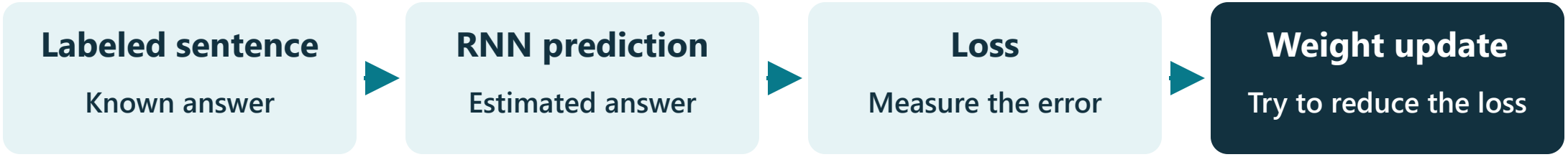
A **Dense layer** combines the final state into one score. **Sigmoid** turns the score into a value from 0 to 1: the estimated positive probability.

We use negative = 0 and positive = 1. Below 0.5 means negative. At least 0.5 means positive. Other tasks may need an output at every step.

> The RNN gathers sequence information. The final layer turns that information into the answer for our chosen task.

7. Training: learn from the error

Loss is a number that measures how far the model's answer is from the known label.



The update reaches the RNN and the embedding, not just the final layer.

Backpropagation through time traces the error backward through the repeated RNN steps. This finds how weights affected the loss.

An **optimizer** changes those weights a little. The same process repeats over training examples. An **epoch** is one pass through the training data.

> During prediction, the weights stay fixed. The hidden state still changes as each new word is read.

8. Long sequences are harder

A simple RNN repeatedly rewrites one small state. Useful earlier information may fade.

Earlier clue

"I did not ..."

The word "not" matters.

Many later words

... long description ...

Many state updates happen.

Last words

"... like this movie."

Earlier clues can fade.

During training, the signal used to improve early steps may become very small. This is called a **vanishing gradient**: an update signal that fades.

RNNs can learn useful short patterns. They do not always keep long-range clues well. LSTM adds another state path and learned gates to help.

> NLP models learn text patterns. Assigning letters or words IDs does not give them human understanding.

9. A small sentence project

We classify a few short, made-up sentences. No large dataset or download is needed.

Train

"i like this movie"

Label: positive = 1.

Test

"we love this game"

Keep separate from training.

Try fresh inputs

"we like this game"

"this movie is awful"

TextVectorization makes the vocabulary from **training text only**. Embedding turns IDs into vectors. SimpleRNN reads them in order.

We check test accuracy, then show predictions for fresh sentences. We also try an unfamiliar word to see the limits of the vocabulary.

> This tiny exercise shows the steps. Its accuracy does not prove that it works on real reviews, sarcasm, or long text.

DAY 10 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie